Football Ticket Booking System - Database Design & SQL Queries

Overview & Objectives

This assignment is designed to evaluate your understanding of database table design, ERD relationships, and intermediate-to-advanced SQL queries. You will work with a simplified **Football Ticket Booking System** database.

By completing this assignment, you will be able to:

- Design an ERD with 1-to-1, 1-to-Many, and Many-to-1 relationships.
- Understand primary keys, foreign keys, and referential integrity.
- Write complex SQL queries using `JOIN` variants, subqueries, aggregations, pattern matching, null handling, and pagination.

Database Design & Business Logic

The system manages football fans purchasing tickets, upcoming tournament matches, and individual ticket booking receipts.

Business Logic & Schema Architecture

Your database design must handle these real-world scenarios by implementing the following exact table fields:

1. Users Table

This table tracks all administrative staff and customers who use the platform.

Field Name	What the Field Does
`user_id`	Unique identification number for each registered user.
`full_name`	Stores the first and last name of the user.
`email`	Stores the user's mail address used for login.
`role`	Defines access permissions (`Ticket Manager` or `Football Fan`).
`phone_number`	Stores the contact mobile number of the fan.

2. Matches Table

This table catalogs the tournament events, stadium logistics, and baseline ticket inventory pricing.

Field Name	What the Field Does
`match_id`	Unique identification number for each football match.
`fixture`	Tracks the two competing teams (e.g., *Real Madrid vs Barcelona*).
`tournament_category`	The league or cup title (e.g., *Champions League*, *Premier League*).
`base_ticket_price`	The foundational price for a single standard entry seat.
`match_status`	Current ticket availability state (`Available` , `Selling Fast` , `Sold Out` , `Postponed`).

3. Bookings Table

This transactional table records individual ticket purchases by linking users to their chosen matches.

Field Name	What the Field Does
`booking_id`	Unique tracking transaction number for the ticket purchase.
`user_id`	Links the booking directly to the user who made the purchase.
`match_id`	Links the booking directly to the specific match being attended.
`seat_number`	The specific allocated seat identifier in the stadium (e.g., `A-12`).
`payment_status`	Tracks financial resolution (`Pending` , `Confirmed` , `Cancelled` , `Refunded`).
`total_cost`	The calculated final invoice price based on the base price and seat quantity.

Part 1: ERD Design (Mandatory)

Note: You must submit your ERD in the submission; otherwise, you will get 0 marks.

Design an Entity Relationship Diagram (ERD) for the Football Ticket Booking System.

Required Tables

You must include the following tables:

1. Users
2. Matches
3. Bookings

Relationship Requirements

Your ERD must clearly show:

- **One to Many:** One User → Many Bookings (A single football fan can buy tickets for multiple matches throughout the season).
- **Many to One:** Many Bookings → One Match (A major derby match can be associated with thousands of individual booking records from different fans).
- **One to One (logical):** Each individual row in the booking table maps exactly one user to one match for that specific reserved seating choice.

ERD Must Include

- Primary Keys (PK)
- Foreign Keys (FK)
- Relationship cardinality (e.g., Crow's Foot notation)
- Status fields (e.g., booking payment status, match ticket status)

Submission Format

You need to submit your ERD as:

- Lucidchart or Draw.io ERD Tool
- Submit a public, shareable ERD link (ensure permissions are set to "Anyone with the link can view").

Schema & Sample Data

1. Users Table

user_id	full_name	email	role	phone_number
1	Tanvir Rahman	tanvir@mail.com	Football Fan	+8801711111111
2	Asif Haque	asif@mail.com	Football Fan	+8801722222222
3	Sajjad Rahman	sajjad@mail.com	Ticket Manager	+8801733333333
4	Jannat Ara	jannat@mail.com	Football Fan	*NULL*

2. Matches Table

match_id	fixture	tournament_category	base_ticket_price	match_status
101	Real Madrid vs Barcelona	Champions League	150	Available

match_id	fixture	tournament_category	base_ticket_price	match_status
102	Man City vs Liverpool	Premier League	120	Selling Fast
103	Bayern Munich vs PSG	Champions League	130	Available
104	AC Milan vs Inter Milan	Serie A	90	Sold Out
105	Juventus vs Roma	Serie A	80	Available

3. Bookings Table

booking_id	user_id	match_id	seat_number	payment_status	total_cost
501	1	101	A-12	Confirmed	150
502	1	102	B-04	Confirmed	120
503	2	101	A-13	Confirmed	150
504	2	101	*NULL*	*NULL*	150
505	3	102	C-20	Pending	120

Part 2: SQL Queries & Expected Sample Output (Open the QUERY.sql file and start your queries from there)

Query 1: Retrieve all upcoming football matches belonging to the 'Champions League' where the match status is 'Available'.

Sample Output:

match_id	fixture	base_ticket_price
101	Real Madrid vs Barcelona	150
103	Bayern Munich vs PSG	130

Query 2: Search for all users whose full names start with 'Tanvir' or contain the phrase 'Haque' (case-insensitive).

- **Concepts used:** `LIKE`, `ILIKE`

Sample Output:

user_id	full_name	email
1	Tanvir Rahman	tanvir@mail.com
2	Asif Haque	asif@mail.com

Query 3: Retrieve all booking records where the payment status is missing (`NULL`), replacing the empty result with 'Action Required'.

- **Concepts used:** `IS NULL , COALESCE`

Sample Output:

booking_id	user_id	match_id	systematic_status
504	2	101	Action Required

Query 4: Retrieve match booking details along with the User's full name and the scheduled Match fixture teams.

- **Concepts used:** `INNER JOIN`

Sample Output:

booking_id	full_name	fixture	total_cost
501	Tanvir Rahman	Real Madrid vs Barcelona	150
502	Tanvir Rahman	Man City vs Liverpool	120
503	Asif Haque	Real Madrid vs Barcelona	150
504	Asif Haque	Real Madrid vs Barcelona	150
505	Sajjad Rahman	Man City vs Liverpool	120

Query 5: Display a comprehensive list of all users and their booking IDs, ensuring that fans who have *never* bought a ticket are still listed.

- **Concepts used:** `LEFT JOIN / Full JOIN`

Sample Output:

user_id	full_name	booking_id
1	Tanvir Rahman	501
1	Tanvir Rahman	502
2	Asif Haque	503
2	Asif Haque	504
3	Sajjad Rahman	505
4	Jannat Ara	*NULL*

Query 6: Find all ticket bookings where the total cost is strictly higher than the average cost of all ticket bookings.

Sample Output:

booking_id	match_id	total_cost
501	101	150
503	101	150
504	101	150

Query 7: Retrieve the top 2 most expensive matches sorted by base ticket price, skipping the absolute highest premium match.

Sample Output:

(Skips Real Madrid vs Barcelona at 150)

match_id	fixture	base_ticket_price
103	Bayern Munich vs PSG	130
102	Man City vs Liverpool	120

Part 3: Theory Questions (Viva Practice - `Answer any 3`)

Question 1: What role does a Foreign Key play in the Bookings table, and how does it safeguard against entering a ticket sale for a match that doesn't exist?

Question 2: Why are we unable to use an aggregate function like `COUNT(booking_id)` inside a standard `WHERE` clause to filter match rows? How does `HAVING` solve this?

Question 3: If a Primary Key column guarantees that all row entries are completely unique, why does the database system also explicitly forbid it from containing a `NULL` value?

Question 4: Imagine a newly registered fan who hasn't bought any match tickets yet. If you run a `LEFT JOIN` linking the Users table (left) to the Bookings table (right), what will the resulting rows look like for that specific fan?

Question 5: What is the difference between a main query and a subquery? In what scenarios would you choose to use a subquery over a standard `JOIN` operation?

🎤 Recording Instructions

1. Use your smartphone selfie camera or laptop webcam in **landscape (horizontal) mode**.
2. Record in a well-lit, quiet room with your face fully visible throughout the video.
3. Select and answer any **3 questions** from the list above, spoken in **English**.
4. Keep each answer between 2–5 minutes**. Speak naturally from your understanding — avoid reading verbatim from notes or scripts.
5. Upload your video to Google Drive, YouTube (Unlisted), or any cloud platform, and share a publicly accessible link.

Evaluation Criteria

README

Section	Marks
ERD Design	40%
SQL Queries	40%
Theory Answers	20%
Total	100%

3️⃣ Final Submission Checklist

Submit the following items via your assignment submission portal:

- ✅ **ERD Link (Public):** Ensure link access configuration is set to public viewer mode.
- ✅ **GitHub Repository Link (Public):** Link to your source code.
- ✅ **Interview Video Link (Public):** Shared via Google Drive, YouTube (Unlisted), or a similar platform.

💡 **Pro Tips:**

- Avoid single-commit repository pushes.
- Double-check that all links are fully accessible externally before submitting.

🎓 Assignment Deadlines

Marks Awarded	Submission Deadline
60 Marks	June 15, 2026 at 11:59 PM
50 Marks	June 16, 2026 at 11:59 PM
30 Marks	June 17 to July 7, 2026 at 11:59 PM

⚠️ **Academic Integrity Policy:** Plagiarism will not be tolerated. All submissions must represent your own

Releases

No releases published

Packages

No packages published

Contributors 1

Eh-haque Ehtisamul Haque